

Project Report — Day 1

Amazon Review NLP & Generative AI Web Application

1. Project Objective

The objective of this project is to develop an interactive web application that analyzes Amazon e-commerce product reviews using Natural Language Processing (NLP) techniques. The system will gradually evolve from basic data analysis to a conversational AI system powered by Retrieval-Augmented Generation (RAG).

The final application aims to:

- Understand customer sentiment
- Extract useful product insights
- Provide summarized information
- Answer user queries using real reviews

Day-1 focuses on environment setup, dataset exploration, and building an initial Streamlit dashboard.

2. Dataset Description

The dataset used is the **Amazon Tech Product Reviews dataset** obtained from Kaggle. It contains real customer reviews for electronic products sold on Amazon.

Main Features

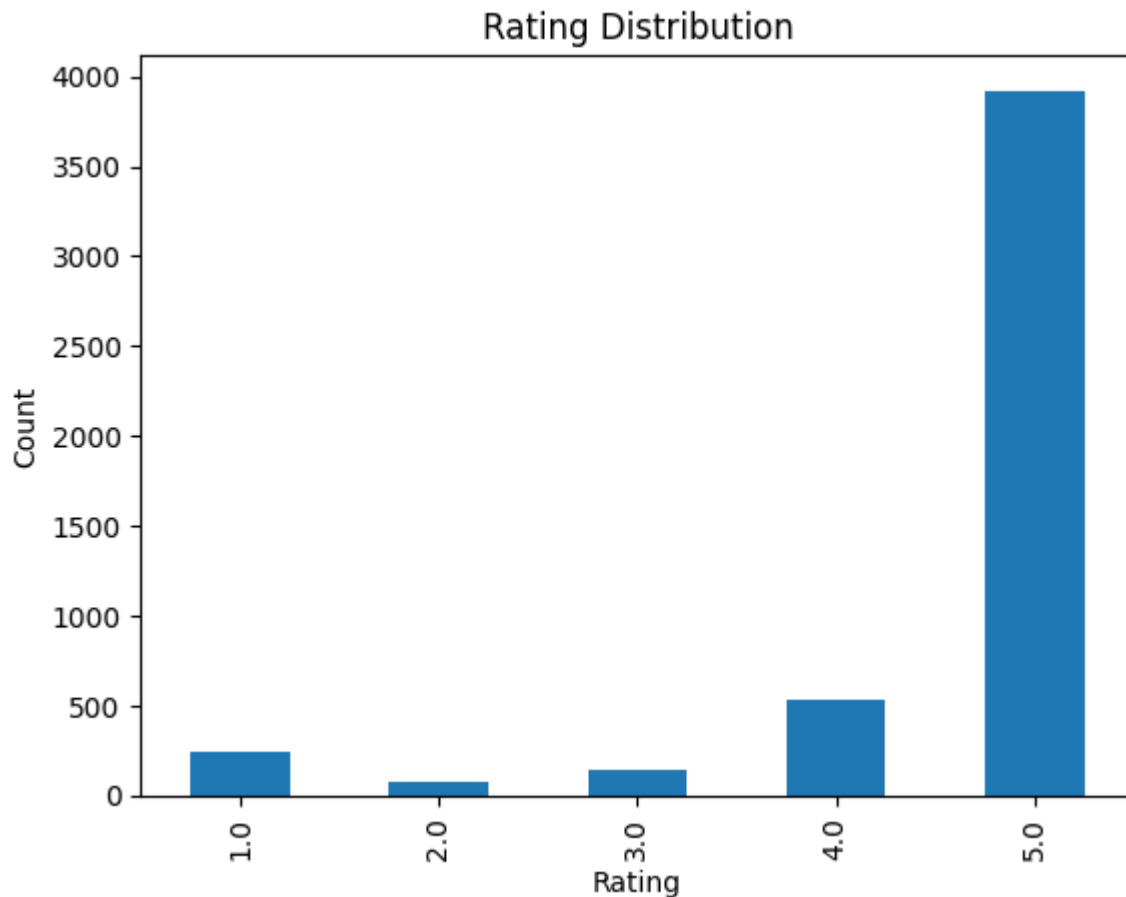
- reviewText — Detailed customer review
- overall — Product rating (1–5 stars)
- summary — Short review title
- asin — Product identifier
- reviewerID — Reviewer identifier
- reviewTime — Date of review

This dataset is suitable for sentiment analysis, text mining, and generative AI applications.

3. Exploratory Data Analysis (EDA)

3.1 Rating Distribution

Analysis of rating values shows that most reviews are rated 4 or 5 stars. This indicates a strong positive bias in customer feedback and introduces class imbalance, which may affect machine learning models in later stages.



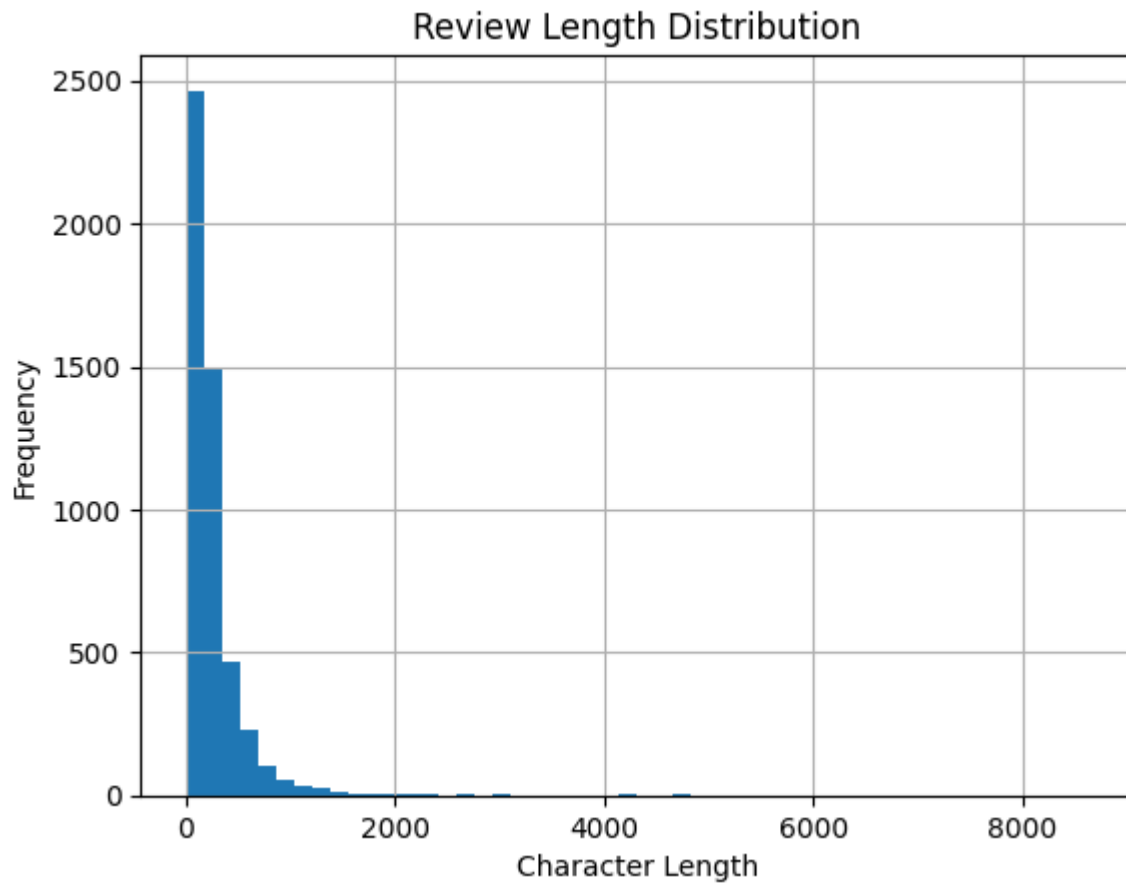
3.2 Review Length Analysis

A new feature `review_length` was created to measure the number of characters in each review.

Observations:

- Review sizes vary significantly
- Some users provide short feedback such as quick opinions
- Others provide detailed experiences and product evaluations

This variation suggests that future NLP processing must handle both short and long text inputs effectively.



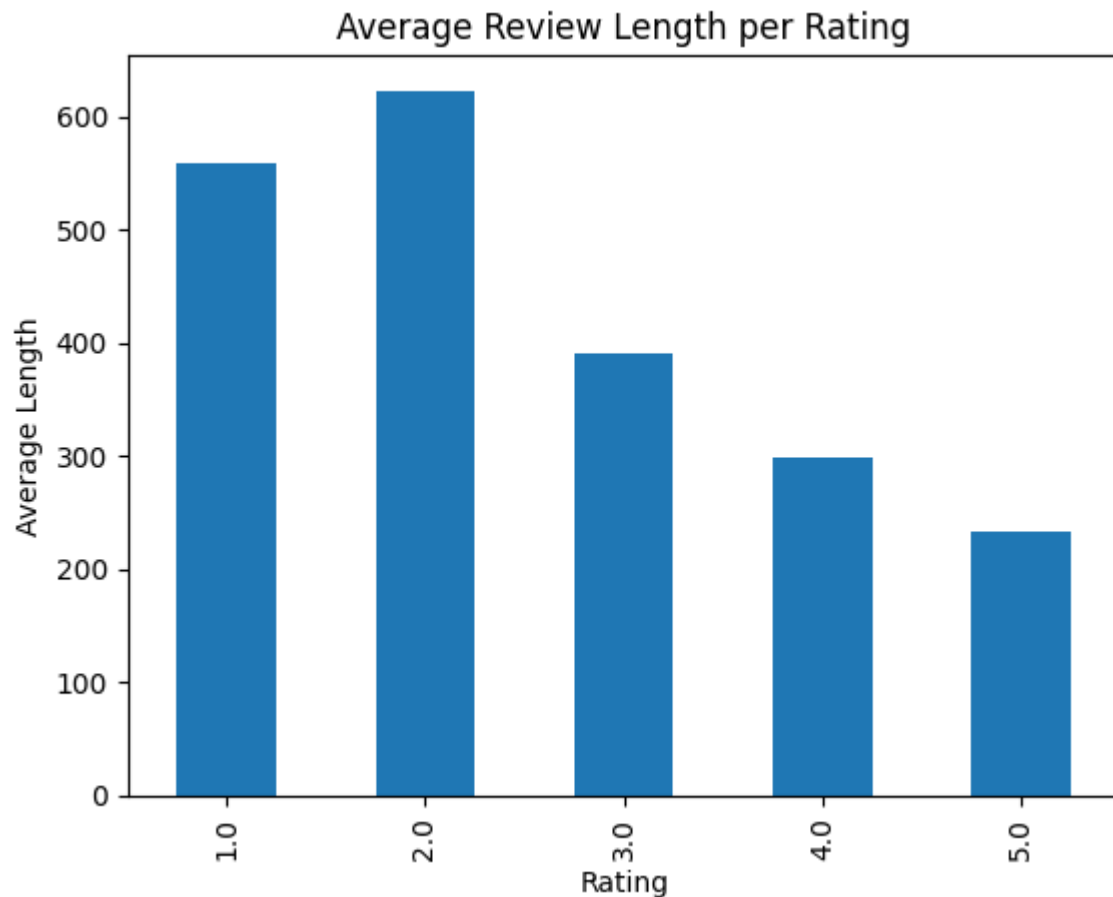
3.3 Rating vs Review Length

The relationship between rating and review length was analyzed.

Key findings:

- Low ratings often have longer reviews because customers explain issues in detail
- High ratings sometimes contain short appreciation comments
- Emotional intensity appears to influence how much users write

This insight will help in feature engineering and also guide chunk size selection when building the future RAG retrieval system.



3.4 Handling Missing Values

Some entries in the reviewText column contained missing values.

They were handled using:

```
fillna("")
```

This ensures stable processing and prevents runtime errors during text analysis.

4. Streamlit Dashboard

A basic Streamlit application was developed to visualize and interact with the dataset.

The dashboard includes:

- Dataset preview table
- Rating distribution chart
- Review length distribution
- Rating vs review length comparison

This serves as the foundation for adding sentiment prediction and AI-based features in later stages.

5. Conclusion

Day-1 successfully established the project environment and performed initial analysis of the dataset. Important behavioral patterns in customer reviews were identified, and a working interactive dashboard was created.

Day 2 – Text Cleaning & Multi-Class Sentiment Model Integration

1. Objective

The goal of Day-2 was to preprocess raw review text and integrate a traditional machine learning model into the Streamlit application for real-time rating prediction. This marks the transition from exploratory analysis to applied NLP modeling.

2. Text Cleaning & Preprocessing

Before training the model, the review text was cleaned to reduce noise and improve feature quality. The following preprocessing steps were applied:

- Converted text to lowercase to ensure uniformity
- Removed special characters, punctuation, and numerical noise
- Removed common English stopwords (e.g., *the*, *is*, *and*)
- Handled missing values using `fillna("")` to prevent runtime errors

These steps help reduce irrelevant information and improve the effectiveness of feature extraction. Clean text ensures that the model focuses on meaningful patterns rather than formatting inconsistencies.

3. Feature Engineering – Why TF-IDF?

TF-IDF (Term Frequency–Inverse Document Frequency) was used to transform textual data into numerical feature vectors.

TF-IDF was selected because:

- It assigns higher importance to distinctive words
- Reduces the influence of frequently occurring but less informative terms
- Works efficiently with large datasets
- Provides strong baseline performance in text classification tasks
- Is computationally lightweight compared to deep learning models

As a baseline method, TF-IDF is widely used in traditional NLP pipelines before transitioning to embeddings or transformer-based approaches.

4. Model Selection – Multinomial Logistic Regression

A multinomial Logistic Regression model was trained to predict review ratings from **1 to 5 stars** instead of binary sentiment.

Logistic Regression was chosen because:

- It performs well with sparse TF-IDF features
- It is fast to train and interpret
- It supports multi-class classification
- It provides probability confidence scores

Class imbalance was addressed using class weighting to improve prediction fairness across rating categories.

5. Application Integration

The trained model and TF-IDF vectorizer were saved as serialized .pkl files and loaded into the Streamlit application.

Users can now:

- Enter custom review text
- Receive predicted star ratings (1–5)
- View confidence probabilities for each rating

This completes the first end-to-end NLP pipeline within the application — from raw text input to real-time prediction.

6. Conclusion

Day-2 successfully transformed the project from exploratory analysis into a functional NLP application. Text preprocessing was implemented, and a multi-class TF-IDF + Logistic Regression model was trained to predict review ratings. The trained model was integrated into the Streamlit interface, enabling real-time user interaction and prediction.